

Maven Movies Rental Business

SQL Data Analysis Project

An End-to-End SQL Portfolio Project

Prepared by:

Payal Rana

Role:

B.Tech Graduate | Aspiring Data Analyst

Tools Used:

SQL (MySQL), MySQL Workbench

Project Type:

Data Analytics Maven Movies Project

Year:

2026

Project Overview

This project applies SQL to analyze operational data for a DVD/movie rental business (the Maven Movies dataset). The analysis works through exploratory data profiling, schema discovery, and a series of realistic ad-hoc business questions that a rental company's marketing, inventory, and management teams might ask.

The goal is to demonstrate practical SQL fluency — joins, aggregation, subqueries, CASE-based segmentation, and set operations — while answering questions with direct business value: which customers to reward, which titles to stock more of, which store performs best, and how to keep the film catalog organized for recommendations and reporting.

Dataset Description

The MAVENMOVIES schema models a multi-store movie rental business, structured as a set of related tables covering customers, films, inventory, rentals, payments, staff, and cast.

Key Tables:

- CUSTOMER / ADDRESS — customer profiles, contact details, and home store assignment
- FILM / CATEGORY / LANGUAGE / FILM_CATEGORY — the film catalog, genres, and language
- ACTOR / FILM_ACTOR — cast members and which films they appear in
- INVENTORY — physical copies of each film, tied to a specific store
- RENTAL — individual rental transactions (who rented what, and when)
- PAYMENT — payments tied to rentals and customers
- STAFF / ADVISOR — employees and advisors associated with the business

The SQL script begins with exploratory SELECT * queries against the core tables to understand structure and relationships, before moving into grouped, filtered, and case-based business analysis.

1 Basic Dataset Overview & EDA

Initial exploration of the schema and the catalog's pricing and rating composition.

1. Provide customer first name, last name, and email ID to the marketing team.

```
SELECT first_name, last_name, email  
FROM CUSTOMER;
```

Explanation: Pulls the core contact fields the marketing team needs for outreach campaigns, without exposing unrelated account or address details.

Result: to be populated by running this query against the live MAVENMOVIES database.

2. How many movies have a rental rate of \$0.99?

```
SELECT COUNT(*) AS CHEAPEST_RENTALS  
FROM film  
WHERE rental_rate = 0.99;
```

Explanation: Counts films at the lowest price tier, useful for understanding how much of the catalog sits in the budget pricing bracket.

Result: to be populated by running this query against the live MAVENMOVIES database.

3. How many movies are in each rental rate category?

```
SELECT rental_rate, COUNT(*) AS total_num_of_movies  
FROM film  
GROUP BY rental_rate;
```

Explanation: Breaks the catalog down by every distinct rental price point, showing how titles are distributed across pricing tiers.

Result: to be populated by running this query against the live MAVENMOVIES database.

4. Which rating has the most films?

```
SELECT RATING, COUNT(*) AS RATING_CATEGORY_COUNT  
FROM FILM  
GROUP BY RATING  
ORDER BY RATING_CATEGORY_COUNT DESC;
```

Explanation: Ranks MPAA rating categories (G, PG, PG-13, R, NC-17) by film count to reveal the dominant content rating in the library.

Result: to be populated by running this query against the live MAVENMOVIES database.

5. Which rating is most prevalent in each store?

```
SELECT I.store_id, F.rating, COUNT(F.rating) AS TOTAL_FILMS  
FROM inventory AS I LEFT JOIN film AS F  
  ON I.film_id = F.film_id  
GROUP BY I.store_id, F.rating  
ORDER BY TOTAL_FILMS DESC;
```

Explanation: Joins inventory to film to see how rating mix differs by physical store location, which can inform localized stocking decisions.

Result: to be populated by running this query against the live MAVENMOVIES database.

6. List of films by film name, category, and language.

```
SELECT F.TITLE, LANG.NAME AS LANGUAGE_NAME, C.NAME AS CATEGORY_NAME
FROM FILM AS F LEFT JOIN LANGUAGE AS LANG
  ON F.LANGUAGE_ID = LANG.LANGUAGE_ID
LEFT JOIN film_category AS FC
  ON F.FILM_ID = FC.FILM_ID
LEFT JOIN CATEGORY AS C
  ON FC.CATEGORY_ID = C.CATEGORY_ID;
```

Explanation: Combines three tables to produce a full catalog view — every title alongside its language and genre — useful as a master reference list.

Result: to be populated by running this query against the live MAVENMOVIES database.

2. Rental Activity & Revenue Analysis

Which titles, customers, and stores drive the most rental activity and revenue.

7. How many times has each movie been rented out?

```
SELECT F.TITLE, COUNT(R.RENTAL_ID) AS POPULARITY
FROM RENTAL AS R LEFT JOIN INVENTORY AS INV
  ON R.INVENTORY_ID = INV.INVENTORY_ID
LEFT JOIN FILM AS F
  ON INV.FILM_ID = F.FILM_ID
GROUP BY F.TITLE
ORDER BY POPULARITY DESC;
```

Explanation: Chains rental → inventory → film to count rentals per title, effectively ranking films by popularity/demand.

Result: to be populated by running this query against the live MAVENMOVIES database.

8. Revenue per film — Top 10 grossing titles.

```
SELECT RENTAL_ID_TRANSACTIONS.TITLE, SUM(P.AMOUNT) AS GROSS_REVENUE
FROM (
  SELECT R.RENTAL_ID, F.FILM_ID, F.TITLE
  FROM RENTAL AS R LEFT JOIN INVENTORY AS INV
    ON R.INVENTORY_ID = INV.INVENTORY_ID
  LEFT JOIN FILM AS F
    ON INV.FILM_ID = F.FILM_ID
) AS RENTAL_ID_TRANSACTIONS
LEFT JOIN PAYMENT AS P
  ON RENTAL_ID_TRANSACTIONS.RENTAL_ID = P.RENTAL_ID
GROUP BY RENTAL_ID_TRANSACTIONS.TITLE
ORDER BY GROSS_REVENUE DESC
LIMIT 10;
```

Explanation: Uses a subquery to map each payment back to its film title, then sums revenue per title to surface the ten highest-grossing movies.

Result: to be populated by running this query against the live MAVENMOVIES database.

9. Which customer has spent the most (for a loyalty reward)?

```
SELECT P.CUSTOMER_ID, SUM(AMOUNT) AS SPENDING, C.FIRST_NAME, C.LAST_NAME
FROM PAYMENT AS P LEFT JOIN CUSTOMER AS C
  ON P.CUSTOMER_ID = C.CUSTOMER_ID
GROUP BY P.CUSTOMER_ID
ORDER BY SPENDING DESC
LIMIT 1;
```

Explanation: Aggregates total payments per customer and returns the single top spender, a natural candidate for a loyalty reward.

Result: to be populated by running this query against the live MAVENMOVIES database.

10. Which store has historically brought the most revenue?

```
SELECT ST.STORE_ID, SUM(P.AMOUNT) AS STORE_REVENUE
FROM PAYMENT AS P LEFT JOIN STAFF AS ST
  ON P.STAFF_ID = ST.STAFF_ID
GROUP BY ST.STORE_ID
ORDER BY STORE_REVENUE DESC;
```

Explanation: Attributes each payment to a store via the staff member who processed it, then totals revenue to compare store performance.

Result: to be populated by running this query against the live MAVENMOVIES database.

11. How many rentals occur each month?

```
SELECT MONTHNAME(RENTAL_DATE) AS MONTH_NAME,
       EXTRACT(YEAR FROM RENTAL_DATE) AS YEAR_NUMBR,
       COUNT(rental.rental_id) AS NUMBER_RENTALS
FROM RENTAL
GROUP BY EXTRACT(YEAR FROM RENTAL_DATE), MONTHNAME(RENTAL_DATE)
ORDER BY NUMBER_RENTALS DESC;
```

Explanation: Breaks rental volume out by month and year, highlighting seasonal peaks and troughs in demand.

Result: to be populated by running this query against the live MAVENMOVIES database.

12. Reward customers who have rented at least 30 times (with contact details).

```
SELECT LOYAL_CUSTOMERS.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME, C.EMAIL, AD.PHONE
FROM (
  SELECT CUSTOMER_ID, COUNT(RENTAL_ID) AS NUMBER_OF_RENTALS
  FROM RENTAL
  GROUP BY CUSTOMER_ID
  HAVING NUMBER_OF_RENTALS >= 30
  ORDER BY CUSTOMER_ID
) AS LOYAL_CUSTOMERS
LEFT JOIN CUSTOMER AS C
  ON LOYAL_CUSTOMERS.CUSTOMER_ID = C.CUSTOMER_ID
LEFT JOIN ADDRESS AS AD
  ON C.ADDRESS_ID = AD.ADDRESS_ID;
```

Explanation: First isolates customers with 30+ lifetime rentals using HAVING, then enriches that list with name, email, and phone for a rewards outreach list.

Result: to be populated by running this query against the live MAVENMOVIES database.

3? Customer Filtering & Engagement

Ad-hoc filtering requests from stakeholders, plus identifying low-engagement customers.

13. Pull all payments from the first 100 customers (by customer ID).

```
SELECT CUSTOMER_ID, RENTAL_ID, AMOUNT, PAYMENT_DATE
FROM PAYMENT
WHERE CUSTOMER_ID < 101;
```

Explanation: A simple range filter on customer ID to scope the payment history to a defined customer cohort.

Result: to be populated by running this query against the live MAVENMOVIES database.

14. Same 100 customers — payments over \$5, since January 1, 2006.

```
SELECT CUSTOMER_ID, RENTAL_ID, AMOUNT, PAYMENT_DATE
FROM PAYMENT
WHERE CUSTOMER_ID < 101
    AND AMOUNT > 5
    AND PAYMENT_DATE > '2006-01-01';
```

Explanation: Layers an amount and date filter on top of the same customer range to narrow in on higher-value, recent transactions.

Result: to be populated by running this query against the live MAVENMOVIES database.

15. Payments from those specific customers, plus any payment over \$5 from any customer.

```
SELECT CUSTOMER_ID, RENTAL_ID, AMOUNT, PAYMENT_DATE
FROM PAYMENT
WHERE AMOUNT > 5
    OR CUSTOMER_ID = 42 OR CUSTOMER_ID = 53
    OR CUSTOMER_ID = 60 OR CUSTOMER_ID = 75;

-- Equivalent, cleaner version using IN():
SELECT CUSTOMER_ID, RENTAL_ID, AMOUNT, PAYMENT_DATE
FROM PAYMENT
WHERE AMOUNT > 5
    AND CUSTOMER_ID IN (42,53,60,75);
```

Explanation: Demonstrates the difference between an OR-based combined filter (broad, matches either condition) and an AND + IN() filter (narrow, matches both), illustrating how logical operators change result scope.

Result: to be populated by running this query against the live MAVENMOVIES database.

16. Which films include a 'Behind the Scenes' special feature?

```
SELECT TITLE, SPECIAL_FEATURES
FROM FILM
WHERE SPECIAL_FEATURES LIKE '%Behind the Scenes%';
```

Explanation: Uses pattern matching on the special_features text column to isolate titles that include bonus behind-the-scenes content.

Result: to be populated by running this query against the live MAVENMOVIES database.

17. Customers with fewer than 15 lifetime rentals (low-engagement outreach list).

```
SELECT CUSTOMER_ID, COUNT(*) AS TOTAL_RENTALS
FROM RENTAL
GROUP BY CUSTOMER_ID
HAVING TOTAL_RENTALS < 15;
```

Explanation: Flags customers below an engagement threshold, useful for win-back campaigns or understanding churn risk.

Result: to be populated by running this query against the live MAVENMOVIES database.

4🔗 Film Catalog Analysis

Understanding pricing, duration, special features, and using CASE logic to segment the catalog.

18. Unique movie ratings and how many movies fall under each.

```
SELECT RATING, COUNT(FILM_ID) AS NUMBER_OF_FILMS
FROM FILM
GROUP BY RATING;
```

Explanation: A straightforward count of titles per MPAA rating, giving a quick content-mix snapshot.

Result: to be populated by running this query against the live MAVENMOVIES database.

19. Count of titles sliced by rental duration.

```
SELECT RENTAL_DURATION, COUNT(FILM_ID) AS NUMBER_OF_FILMS
FROM FILM
GROUP BY RENTAL_DURATION;

-- Cross-tabbed by rating and rental duration together:
SELECT RATING, RENTAL_DURATION, COUNT(FILM_ID) AS NUMBER_OF_FILMS
FROM FILM
GROUP BY RATING, RENTAL_DURATION;
```

Explanation: Shows how many films are assigned to each allowed rental period, then cross-tabs that against rating to see if certain ratings skew toward longer or shorter rental windows.

Result: to be populated by running this query against the live MAVENMOVIES database.

20. Rating, film count, length range, and average rental duration — compared.

```
SELECT RATING,
       COUNT(FILM_ID) AS COUNT_OF_FILMS,
       MIN(LENGTH) AS SHORTEST_FILM,
       MAX(LENGTH) AS LONGEST_FILM,
       AVG(LENGTH) AS AVERAGE_FILM_LENGTH,
       AVG(RENTAL_DURATION) AS AVERAGE_RENTAL_DURATION
FROM FILM
```

```
GROUP BY RATING
ORDER BY AVERAGE_FILM_LENGTH;
```

Explanation: Combines five aggregates in one pass to check whether longer films (by rating category) are also given longer rental windows.

Result: to be populated by running this query against the live MAVENMOVIES database.

21. Does a higher replacement cost correlate with a higher rental rate?

```
SELECT REPLACEMENT_COST,
       COUNT(FILM_ID) AS NUMBER_OF_FILMS,
       MIN(RENTAL_RATE) AS CHEAPEST_RENTAL,
       MAX(RENTAL_RATE) AS EXPENSIVE_RENTAL,
       AVG(RENTAL_RATE) AS AVERAGE_RENTAL
FROM FILM
GROUP BY REPLACEMENT_COST
ORDER BY REPLACEMENT_COST;
```

Explanation: Groups films by replacement cost and summarizes their rental rate range, testing whether pricier-to-replace inventory is priced higher to rent.

Result: to be populated by running this query against the live MAVENMOVIES database.

22. Film titles, lengths, and rental rates — longest to shortest.

```
SELECT TITLE, LENGTH, RENTAL_RATE
FROM FILM
ORDER BY LENGTH DESC
LIMIT 20;
```

Explanation: A simple ranked list checking whether the longest films also carry the highest rental rates.

Result: to be populated by running this query against the live MAVENMOVIES database.

23. Categorize films by length bucket.

```
SELECT TITLE, LENGTH,
       CASE
         WHEN LENGTH < 60 THEN 'UNDER 1 HR'
         WHEN LENGTH BETWEEN 60 AND 90 THEN '1 TO 1.5 HRS'
         WHEN LENGTH > 90 THEN 'OVER 1.5 HRS'
         ELSE 'ERROR'
       END AS LENGTH_BUCKET
FROM FILM;
```

Explanation: Uses a CASE expression to bucket every film into a runtime category, making the catalog easier to browse or filter by duration.

Result: to be populated by running this query against the live MAVENMOVIES database.

24. Categorize films for child/family recommendation fitness.

```
SELECT DISTINCT TITLE,
       CASE
         WHEN RENTAL_DURATION <= 4 THEN 'RENTAL TOO SHORT'
         WHEN RENTAL_RATE >= 3.99 THEN 'TOO EXPENSIVE'
         WHEN RATING IN ('NC-17', 'R') THEN 'TOO ADULT'
         WHEN LENGTH NOT BETWEEN 60 AND 90 THEN 'TOO SHORT OR TOO LONG'
```



```

        WHEN DESCRIPTION LIKE '%Shark%' THEN 'NO_NO_HAS_SHARKS'
        ELSE 'GREAT_RECOMMENDATION_FOR_CHILDREN'
    END AS FIT_FOR_RECOMMENDATION
FROM FILM;

```

Explanation: Applies a multi-condition CASE statement to auto-tag films as suitable or unsuitable for a children's recommendation list, based on duration, price, rating, length, and even keyword screening in the description.

Result: to be populated by running this query against the live MAVENMOVIES database.

5? Store & Customer Status Reporting

Labeling customer accounts by store/active status, and auditing inventory per store.

25. Which store each customer belongs to, and whether their account is active.

```

SELECT CUSTOMER_ID, FIRST_NAME, LAST_NAME,
       CASE
           WHEN STORE_ID = 1 AND ACTIVE = 1 THEN 'store 1 active'
           WHEN STORE_ID = 1 AND ACTIVE = 0 THEN 'store 1 inactive'
           WHEN STORE_ID = 2 AND ACTIVE = 1 THEN 'store 2 active'
           WHEN STORE_ID = 2 AND ACTIVE = 0 THEN 'store 2 inactive'
           ELSE 'ERROR'
       END AS STORE_AND_STATUS
FROM CUSTOMER;

```

Explanation: Combines store assignment and active flag into one readable label per customer, useful for account-status reporting.

Result: to be populated by running this query against the live MAVENMOVIES database.

26. Full inventory list — film title, description, store, and inventory ID.

```

SELECT DISTINCT INVENTORY.INVENTORY_ID,
               INVENTORY.STORE_ID,
               FILM.TITLE,
               FILM.DESCRPTION
FROM FILM
INNER JOIN INVENTORY ON FILM.FILM_ID = INVENTORY.FILM_ID;

```

Explanation: Joins film to inventory to produce a per-copy listing that shows exactly what's in stock at which store.

Result: to be populated by running this query against the live MAVENMOVIES database.

6? Actor & Cast Analysis

Understanding cast size per film and building an actor-to-filmography lookup.

27. Actor first name, last name, and number of films they've appeared in.

```

SELECT
    ACTOR.ACTOR_ID,
    ACTOR.FIRST_NAME,

```

```

    ACTOR.LAST_NAME,
    COUNT(FILM_ACTOR.FILM_ID) AS NUMBER_OF_FILMS
FROM ACTOR
LEFT JOIN FILM_ACTOR
    ON ACTOR.ACTOR_ID = FILM_ACTOR.ACTOR_ID
GROUP BY ACTOR.ACTOR_ID;

```

Explanation: Joins the actor and film_actor bridge table to count filmography size for every actor in the database.

Result: to be populated by running this query against the live MAVENMOVIES database.

28. How many actors are associated with each film title (for an investor).

```

SELECT FILM.TITLE,
    COUNT(FILM_ACTOR.ACTOR_ID) AS NUMBER_OF_ACTORS
FROM FILM
LEFT JOIN FILM_ACTOR
    ON FILM.FILM_ID = FILM_ACTOR.FILM_ID
GROUP BY FILM.TITLE;

```

Explanation: The inverse of the previous query — counts cast size per title, giving investors a sense of production scale across the catalog.

Result: to be populated by running this query against the live MAVENMOVIES database.

29. Full list of actors and every title they appear in.

```

SELECT ACTOR.FIRST_NAME,
    ACTOR.LAST_NAME,
    FILM.TITLE
FROM ACTOR
INNER JOIN FILM_ACTOR
    ON ACTOR.ACTOR_ID = FILM_ACTOR.ACTOR_ID
INNER JOIN FILM
    ON FILM_ACTOR.FILM_ID = FILM.FILM_ID
ORDER BY ACTOR.LAST_NAME, ACTOR.FIRST_NAME;

```

Explanation: Chains actor → film_actor → film to build a customer-facing 'actor filmography' lookup, sorted alphabetically by actor name.

Result:

	FIRST_NAME	LAST_NAME	TITLE
▶	CHRISTIAN	AKROYD	BACKLASH UNDEFEATED
	CHRISTIAN	AKROYD	BETRAYED REAR
	CHRISTIAN	AKROYD	CAPER MOTIONS
	CHRISTIAN	AKROYD	CATCH AMISTAD
	CHRISTIAN	AKROYD	CHANCE RESURRECTION
	CHRISTIAN	AKROYD	CONFUSED CANDLES
	CHRISTIAN	AKROYD	CUPBOARD SINNERS
	CHRISTIAN	AKROYD	DIVIDE MONSTER
	CHRISTIAN	AKROYD	DOOM DANCING
	CHRISTIAN	AKROYD	DOORS PRESIDENT
	CHRISTIAN	AKROYD	DRIVER ANNIE
	CHRISTIAN	AKROYD	FEATHERS METAL
	CHRISTIAN	AKROYD	EDGE WOLVES

Sample output (sorted by last name): actor Christian Akroyd's filmography, showing every title he appears in.

7? Store Expansion & Staff Reporting

Supporting collection-expansion decisions at Store 2, and combining staff/advisor rosters.

30. Distinct titles and descriptions currently in inventory at Store 2 (for collection expansion planning).

```
SELECT DISTINCT FILM.TITLE,  
               FILM.DESCRPTION  
FROM FILM  
INNER JOIN INVENTORY  
  ON FILM.FILM_ID = INVENTORY.FILM_ID  
 AND INVENTORY.STORE_ID = 2;
```

Explanation: Filters the catalog to only what's physically stocked at Store 2, giving the store manager a clear view of the current assortment before deciding what to add.

Result: to be populated by running this query against the live MAVENMOVIES database.

31. Combined staff and advisor directory, labeled by role, for an all-hands meeting.

```
(SELECT FIRST_NAME, LAST_NAME, 'ADVISORS' AS DESIGNATION  
 FROM ADVISOR)  
UNION  
(SELECT FIRST_NAME, LAST_NAME, 'STAFF MEMBER' AS DESIGNATION  
 FROM STAFF);
```

Explanation: Uses UNION to stack two separate people tables (staff and advisors) into a single roster, tagging each row with its source so both groups can be printed on one meeting list.

Result: to be populated by running this query against the live MAVENMOVIES database.

🔍 Key Findings & Business Takeaways

Catalog & Pricing

- Rental rate, rating, and rental-duration breakdowns give a full picture of how the catalog is priced and organized.
- The 'Behind the Scenes' special-feature filter and the length/recommendation CASE logic show how the catalog can be auto-tagged for merchandising and family-friendly recommendations.

Rentals & Revenue

- Film popularity (rental counts) and top-10 grossing titles identify which inventory is working hardest for the business.
- Revenue can be attributed by store (through staff), surfacing which location is the stronger performer historically.
- Monthly rental counts expose seasonal demand patterns that can guide staffing and promotions.

Customers

- The top spender and the 30+ rental 'loyal customer' segment are natural targets for a rewards program, complete with contact details for outreach.
- Customers with fewer than 15 lifetime rentals form a win-back / re-engagement list.
- Store/active-status labeling gives a quick account-health view per customer.

Cast & Inventory

- Actor-to-film and film-to-cast-size queries support a customer-facing 'browse by actor' feature and give investors visibility into production scale.
- The Store 2 inventory query directly supports the manager's collection-expansion planning by showing exactly what's already on the shelf.

***Note on figures:** this report documents the query logic and business intent behind each question. Actual numeric results depend on running the script against a live MAVENMOVIES database — run the queries in MySQL Workbench (or supply exported results) to populate the Result lines above with real figures.*

✓ Conclusion

This SQL-based analysis of the Maven Movies rental business demonstrates how relational data across customers, inventory, rentals, payments, and cast can be joined and aggregated to answer real operational questions — from identifying top customers and top-performing stores, to segmenting the film catalog and supporting inventory expansion decisions.

The project covers a broad range of SQL techniques: multi-table joins, subqueries, GROUP BY/HAVING aggregation, CASE-based classification, LIKE pattern matching, and UNION for combining record sets — mirroring the kind of ad-hoc analysis a Data Analyst is asked to perform for marketing, inventory, and management stakeholders.

Together with the accompanying Amazon Sales SQL project, this work showcases the ability to translate raw relational data into decision-ready insights across two different business domains.